1. Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

- Input: m=2,n=2,N=2,i=0,j=0 · Output: 6
- Input: m=1,n=3,N=3,i=0,j=1 · Output: 12

main.c

```
                                Online C Compiler.
                        Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

*******************************************************************************/
#include <stdio.h>

int findPaths(int m, int n, int N, int i, int j)
{
    if(i < 0 || i >= m || j < 0 || j >= n)
        return 1;

    if(N == 0)
        return 0;

    return findPaths(m,n,N-1,i+1,j) +
           findPaths(m,n,N-1,i-1,j) +
           findPaths(m,n,N-1,i,j+1) +
           findPaths(m,n,N-1,i,j-1);
}

int main()
{
    int m,n,N,i,j;

    printf("Enter m n N i j: ");
    scanf("%d%d%d%d%d",&m,&n,&N,&i,&j);

    printf("Number of Paths = %d\n",
           findPaths(m,n,N,i,j));

    return 0;
}
```

Input

```
Enter m n N i j: 2 5 6 2 7
Number of Paths = 1


..Program finished with exit code 0
Press ENTER to exit console.
```

2. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

(i) Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

main.c

```c
#include <stdio.h>

int robLinear(int nums[], int start, int end)
{
    int prev1 = 0, prev2 = 0;

    for(int i = start; i <= end; i++)
    {
        int temp = prev1;

        if(prev2 + nums[i] > prev1)
            prev1 = prev2 + nums[i];

        prev2 = temp;
    }

    return prev1;
}

int rob(int nums[], int n)
{
    if(n == 1)
        return nums[0];

    int case1 = robLinear(nums, 0, n - 2);
    int case2 = robLinear(nums, 1, n - 1);

    return (case1 > case2) ? case1 : case2;
}

int main()
{
    int n;
```

input

```
iter money in each house:
0000
1947
94749
557465
18
aximum money robbed = 1612412


.Program finished with exit code 0
ess ENTER to exit console.
```

3. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

(i) Input: n=4 Output: 5

(ii) Input: n=3 Output: 3

```c
#include <stdio.h>

int climbStairs(int n)
{
    if(n == 1)
        return 1;

    int first = 1, second = 2;

    for(int i = 3; i <= n; i++)
    {
        int third = first + second;
        first = second;
        second = third;
    }

    return second;
}

int main()
{
    int n;

    printf("Enter number of steps: ");
    scanf("%d", &n);

    printf("Number of ways = %d\n", climbStairs(n));

    return 0;
}
```

```
input
Enter number of steps: 2
Number of ways = 2


..Program finished with exit code 0
Press ENTER to exit console.
```

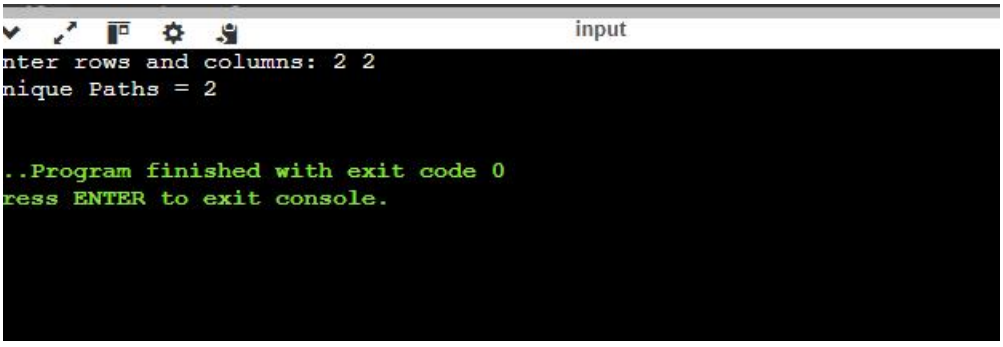
4. A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

(i) Input: m=7,n=3 Output: 28

(ii) Input: m=3,n=2 Output: 3

```c
#include <stdio.h>

int uniquePaths(int m, int n)
{
    int dp[100][100];

    for(int i = 0; i < m; i++)
        dp[i][0] = 1;

    for(int j = 0; j < n; j++)
        dp[0][j] = 1;

    for(int i = 1; i < m; i++)
    {
        for(int j = 1; j < n; j++)
        {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }

    return dp[m-1][n-1];
}

int main()
{
    int m, n;

    printf("Enter rows and columns: ");
    scanf("%d%d", &m, &n);

    printf("Unique Paths = %d\n", uniquePaths(m, n));
```



5. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

   Example 1:
   Input: s = "abbxxxxzzy"
   Output: [[3,6]]
   Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:
Input: s = "abc"

ain.c

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s[100];

    printf("Enter string: ");
    scanf("%s", s);

    int n = strlen(s);

    printf("Large Groups: ");

    int found = 0;

    for(int i = 0; i < n;)
    {
        int start = i;

        while(i < n && s[i] == s[start])
            i++;

        int end = i - 1;

        if(end - start + 1 >= 3)
        {
            printf("[%d,%d] ", start, end);
            found = 1;
        }
    }

```

input

```
Enter string: skfnjb
Large Groups: []


...Program finished with exit code 0
Press ENTER to exit console.
```

Output: []
Explanation: We have groups "a", "b", and "c", none of which are large groups.

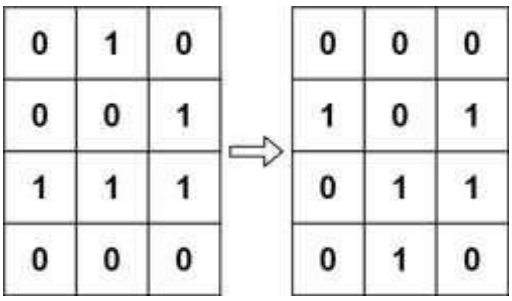
6. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

   Any live cell with fewer than two live neighbors dies as if caused by under-population.

   1. Any live cell with two or three live neighbors lives on to the next generation.
   2. Any live cell with more than three live neighbors dies, as if by over-population.
   3. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

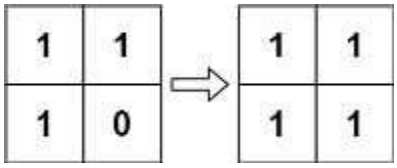
   The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return *the next state*.

   Example 1:



   Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
   Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

   Example 2:



   Input: board = [[1,1],[1,0]]
   Output: [[1,1],[1,1]]

main.c

```c
#include <stdio.h>

int main()
{
    int m, n;

    printf("Enter rows and columns: ");
    scanf("%d%d", &m, &n);

    int board[50][50];
    int next[50][50];

    printf("Enter board elements:\n");

    for(int i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
        {
            scanf("%d",&board[i][j]);
        }
    }

    int dir[8][2] = {
        {-1,-1}, {-1,0}, {-1,1},
        {0,-1},          {0,1},
        {1,-1},  {1,0},  {1,1}
    };

    for(int i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
        {
            int live = 0;
```

input

```
Enter rows and columns: 2 2
Enter board elements:
2 5
2
5

Next State:
0 0
0 0
```

7. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the

second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.

Now after pouring some non-negative integer cups of champagne, return how full the j^{th} glass in the i^{th} row is (both i and j are 0-indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1
Output: 0.00000
Explanation: We poured 1 cup of champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

Example 2:

Input: poured = 2, query_row = 1, query_glass = 1
Output: 0.50000
Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

```c
#include <stdio.h>

double champagneTower(int poured, int query_row, int query_glass)
{
    double dp[101][101] = {0};

    dp[0][0] = poured;

    for(int i = 0; i < 100; i++)
    {
        for(int j = 0; j <= i; j++)
        {
            if(dp[i][j] > 1.0)
            {
                double excess = (dp[i][j] - 1.0) / 2.0;
                dp[i + 1][j] += excess;
                dp[i + 1][j + 1] += excess;
            }
        }
    }

    if(dp[query_row][query_glass] > 1.0)
        return 1.0;

    return dp[query_row][query_glass];
}

int main()
{
    int poured, query_row, query_glass;

    printf("Enter poured cups: ");
    scanf("%d", &poured);
```

input

```
Enter poured cups: 5
Enter query row: 2
Enter query glass: 1
Output: 1.00000


...Program finished with exit code 0
Press ENTER to exit console.
```